

MLOps Inference Platform: Designing a Scalable Architecture

TABLE OF CONTENTS

1 General Architecture	3
2 Hardware Requirements.....	4
3 Used technologies	5
4 Components	8
5 Types of host filesystem access.....	9
Local volumes.....	9
Network storage.....	10
6 SFTP Server Setup	10
7 ML Ops implementations and K8s Manifests	11
8 Monitoring & Logging (EFK & Prometheus).....	13
9 Operational Strategies	14

1 GENERAL ARCHITECTURE

At its core, the environment is operated via AWS Container Engine (managed Kubernetes cluster). While there is a separation of hosts for applications connected directly to the public internet, e.g. ingress controllers and backend applications, a direct application-to-machine mapping is not possible due to the cluster-approach.

Applications in general will be referred to as “microservices” in this chapter, to reflect this. A microservice typically consists of an application that is packaged into a Docker container which can be configured via environment parameters, or retrieve configuration directly from Vault. In the cluster context, components that interact closely to provide a specific feature or functionality for the host model product are grouped into namespaces.

While most services within the cluster do not required direct internet access, it also consists of some services that integrate directly with external customer APIs and therefore require internet access. Due to security regulations, the access of resources on public internet is only possible indirectly via an internal forward proxy, where outgoing communication paths are required to be whitelisted.

The model host endpoints are publicly accessible via multiple domains/endpoints. This includes application, as well as monitoring and administration endpoints.

To nurture scalability central components of the application environment are only accessible over the internet via Elastic Loadbalancer. Within the cluster Kubernetes provides loadbalancing capabilities by using Service Discovery for communication between the components. Microservices are never exposed directly, but instead rely on Ingress Controllers to provide external access, which in turn usually are located behind ELBs. Combining those features provides an easy and effective way to scale the application.

Most components within the cluster can be setup fully automated, which enables the operation team to setup completely new environments quickly. Although not fully automated, the deployment process is constantly improved to reduce the manual efforts required for performing standard changes, e.g. updating components.

Although OS access is usually not required in this cluster architecture, as AWS or the on prem servers in general act as mere kubernetes-node that are fully managed via EKS, it is possible to access hosts directly. System accounts are managed by a central identity provider service and need to be accessed via AWS console. This level of access is limited to 2nd level support (operations team).

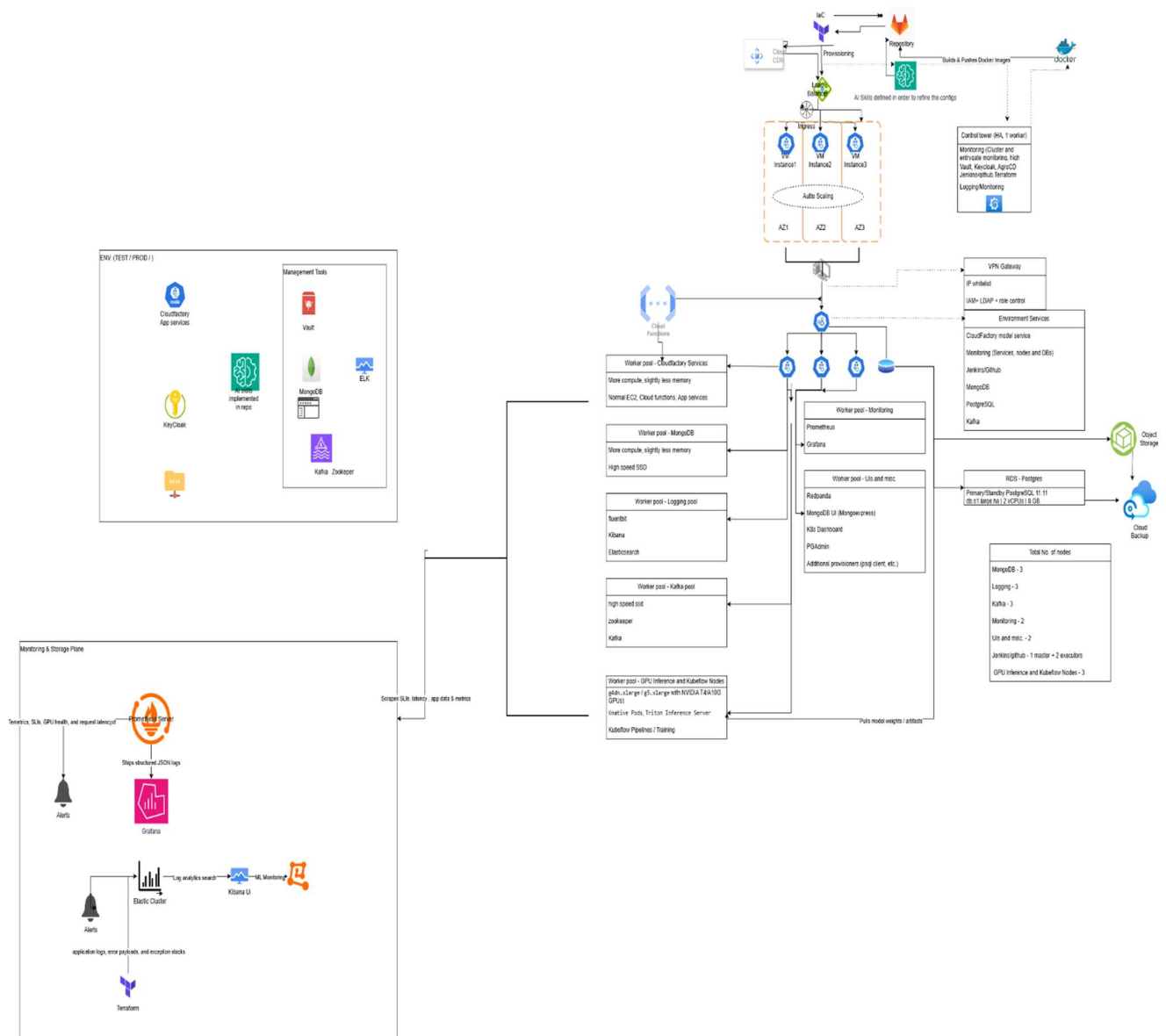


Diagram to the architecture- [Link](#)

https://app.diagrams.net/#G11qYk04_QolaIlxBDYtLl9lOeNZSb7zuW#%7B%22pageId%22%3A%22e-fee-NbQjugqaIYYZdZ%22%7D

2 HARDWARE REQUIREMENTS

As infrastructure resources are constantly improved and updated in AWS Cloud, hardware resources are not strictly documented here. Instead, general concepts of every host model requirement are listed here:

- Compute Resources with a few exceptions are not managed directly, but indirectly as EKS Kubernetes Cluster nodes. The aim is to maintain mostly identical flavors as Node Pools with EKS, that can be scaled up or down, as required by the application. With this flavor sizes are unified and the environment is kept manageable. Another benefit of deploying the application only onto EC2 servers

that are part of EKS node pools is, that instead of maintaining and regularly patching all servers individually, old nodes of a pool can be directly replaced with new (fully updated) nodes via built in K8S mechanisms, making long-lasting and error-prone OS patch installations obsolete.

- Whenever applicable and economically feasible, managed services are used in favor of self-hosted solutions for infrastructure elements, like Databases, or logging solutions. This only applies to managed services, where AWS is able to provide up-to-date solutions.
- Each environment has at least one EC2 with EIP attached that is designated as entrygate-server to the environment, in case direct access to the servers is required.

3 USED TECHNOLOGIES

The following technical components are used to setup and operate the environment.

Software product	Purpose	Description
Docker	Application Deployment	All application components (microservices) are packaged into separate Docker containers. Configuration is possible via environment variables.
Kubernetes	Application Deployment/Operation	Manages all EC2 servers as Kubernetes nodes and deploys containers in environment.
AWS Deployment server	Application Deployment	AWS-managed private Docker registry. Host majority of container images that are deployed to the environment.
Jenkins	Application Deployment	Automation of release deployment, configuration updates, Kafka topic creation and endpoint exposure. Configuration is persisted via Jenkinsfile.
Gitlab	Configuration Storage	Environment specific configuration files for Jenkins and Docker images are persisted here. Contains also custom developed scripts, Prometheus Exporters and other ELK tools.
MySQL DB / Postgre SQL DB	Data persistency	Main relational SQL database in environment. Provided by AWS Relational Database Service.
Mongo DB	Data persistency	Main No-SQL database in environment. Older versions (3.4) are provided by

Software product	Purpose	Description
		AWS Database service, while newer versions (>4.0) are self-hosted.
ELK	Monitoring	Central logging is implemented via Elasticsearch, FluentD and Kibana setup.
Prometheus/Grafana	Monitoring	Prometheus tracks performance metrics from every EKS and container in the environment. Grafana is used to visualize collected data.
NGINx/Traefik	Environment access	Main ingress controllers. They function as reverse proxy and handle SSL termination for incoming requests.
Let's Encrypt	Environment access	In all environments Let's Encrypt is used to provide SSL encryption for all application product domains.

List of Platform Components (Different components which are deployed not as Application related Component)

Component	Purpose	Description	Ops documentation	Helm Chart Version	Image Version
Keycloak	Single Sign On	Provide SSO for customers and Cloudfactory		keycloak-18.1.1	
Jenkins or GitHub	CI/CD	Used for build and deployment		jenkins-2.56.3	
Kafka				kafka-16.0.1	docker.io/bitnami/kafka:3.1.0-debian-10-r0

Component	Purpose	Description	Ops documentation	Helm Chart Version	Image Version
Zookeeper				zookeeper-8.0.1	docker.io/bitnami/zookeeper:3.7.0-debian-10-r265
Vault				vault-0.15.0	hashicorp/vault:1.8.1
Redis				redis-17.1.4	docker.io/bitnami/redis:6.2.6-debian-10-r120
MongoDB				mongodb-10.31.5	docker.io/bitnami/mongodb:4.2
Prometheus				kube-prometheus-stack-34.9.0	quay.io/prometheus/prometheus:v2.34.0
Grafana				kube-prometheus-stack-34.9.0	grafana/grafana:8.1.2
Alert Manager				kube-prometheus-stack-34.9.0	quay.io/prometheus/alertmanager:v0.24.0
Fluentbit				fluent-bit-0.19.22	
Elasticsearch				elasticsearch-8.14.0	
Kibana				kibana-7.14.0	
Ingress (Nginx)				ingress-nginx-4.0.18	

Component	Purpose	Description	Ops documentation	Helm Chart Version	Image Version
Ngapp					
Ingress (Nginx)				ingress-nginx-4.0.16	
AWS Terraform provider	AWS resources	API calls towards AWS	https://registry.terraform.io/providers/hashicorp/aws/latest/docs		To manage this AWS MLOps infrastructure cleanly and reproducibly, you can organize infrastructure code into modular Terraform modules. This approach separates concerns between networking, cluster orchestration, dynamic compute provisioning (Karpenter), and model artifact storage (S3).

4 COMPONENTS

Components of the system:

Component	Kubernetes Namespace	AWS resource	Description
Ingress LB	Ingress LB #1	EC2	HTTP/HTTPS Reverse proxy for access to REST services of Host model app product
Ingress LB	Ingress LB #2	EC2	HTTP/HTTPS Reversy proxy for access to monitoring tools
Keycloak	Keycloak	EC2	Keycloak identity and authentication management
Elasticsearch	ELK		Components to aggregate logfiles for all application parts and searching
Kibana	ELK	EC2	

Component	Kubernetes Namespace	AWS resource	Description
Prometheus	Prometheus	EC2	Components for collecting system metrics and performing check for system monitoring
Grafana	Prometheus	EC2	
GPU and Kubeflow	Kubeflow	EC2	Use central Amazon S3 bucket and configure workflow default node selectors to target GPU worker pools during heavy training jobs.
MySQL/PostgreSQL DB		RDS	Central MySQL database for various components
Mongo DB		EC2	Central Mongo database for various components
Kafka		EC2	Central messaging bus
ObjectStore		S3	Storage for CloudArchive

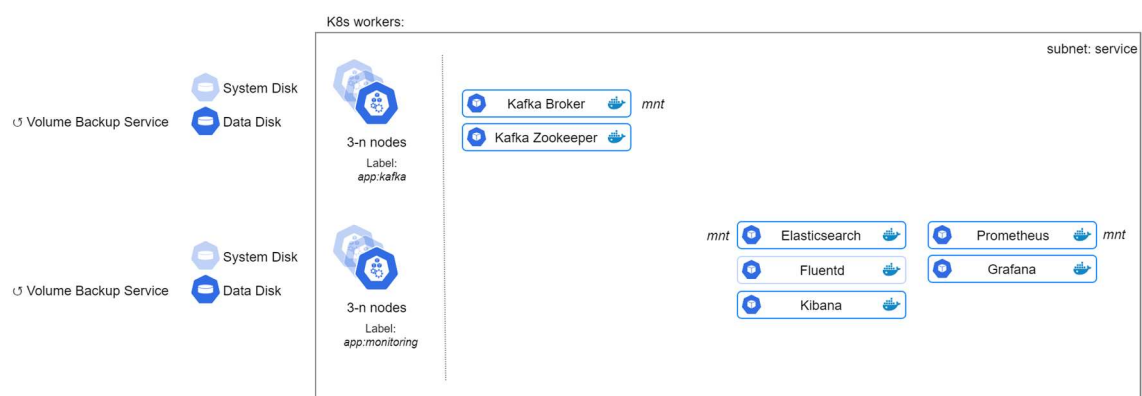
All components listed in the table are hosted on virtual resources in the AWS and on prem. The operating system for all EC2 is Linux.

5 TYPES OF HOST FILESYSTEM ACCESS

Some pods will require access to a filesystem mounted to the host node to either read data from shared configuration files or to persist data.

LOCAL VOLUMES

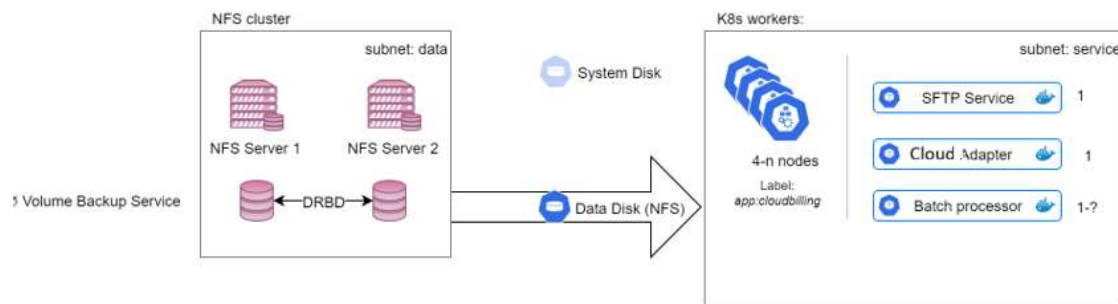
Infrastructure services that require fast access to persisted storage, are hosted on servers with additionally attached local data disks. As an example this applies to log-storage for Kafka brokers, or index-storage for monitoring and logging services within Host model app.



NETWORK STORAGE

While with EFS(Elastic file system) the AWS offers network storage that is scalable and can be attached to any POD directly, it comes with limitations: Slowness and missing backup mechanisms. Those two factors are a showstopper for using EFS to store any non-reproducible data.

Instead a NFS cluster is used to satisfy high-availability requirements as well as to provide adequate read-/write-performance. In addition it is also possible to used standard AWS EVS backup mechanisms to backup any data store in the file-share.



A single node setup without redundancy or replication is established to provide File-Shares to all Docker containers in the K8s network. The file-system is also backed up via AWS EBS.

6 SFTP SERVER SETUP

The SFTP server is hosted directly on a dedicated EC2 server that resides in its own subnet and uses its own security group. Access to the EC2 is possible from the EIP, although SourceIPs allowed to access the server are maintained directly in the Security Group to limit the amount of potential origins. FTP users are chrooted and access to the filesystems is limited by POSIX ACLs.

The server is not connected to central IPA(Identity, Policy, and Audit) user management to mitigate the risk of gaining further access in case of a security incident. System logfiles are however transferred via syslog to a central log-server.

7 ML OPS IMPLEMENTATIONS AND K8S MANIFESTS

THIS MANIFEST DEFINES A MODEL DEPLOYMENT THAT SCALES BASED ON CONCURRENT REQUESTS, LEVERAGING AN S3-BACKED MODEL URI AND REQUESTING GPU HARDWARE ACCELERATION.

```
> Users > user > Downloads > ! infra.yaml
1  apiVersion: "serving.kserve.io/v1beta1"
2  kind: "InferenceService"
3  metadata:
4    name: "customer-churn-model"
5    namespace: "ml-inference"
6    annotations:
7      autoscaling.knative.dev/min-scale: "0"
8      autoscaling.knative.dev/max-scale: "5"
9  spec:
10   predictor:
11     model:
12       storageUri: "s3://company-mlops-model-registry/churn-model/v1.2.0/"
13       modelFormat:
14         name: triton
15       resources:
16         requests:
17           cpu: "2000m"
18           memory: "4Gi"
19         limits:
20           cpu: "4000m"
21           memory: "8Gi"
22       nvidia.com/gpu: "1" # Automatically schedules onto an EKS GPU node pool
```

Building and deploying stage of the ML models using Github.

```

name: Build and Deploy ML Model

on:
  push:
    branches: [ "main" ]
    paths:
      - 'models/**'

env:
  AWS_REGION: eu-central-1
  ECR_REPOSITORY: customer-churn-detector

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build, tag, and push image to Amazon ECR
        env:
          REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG: ${ github.sha }
        run: |
          docker build -t $REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG models/
          docker push $REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG

```

To provision GPU-accelerated nodes dynamically with Karpenter on AWS EKS, you need two custom resources: an EC2NodeClass (which handles AWS-specific configurations like subnets, security groups, and Amazon Machine Image selection) and a NodePool (which defines instance families, capacity types like Spot vs. On-Demand, and scaling rules).

8 MONITORING & LOGGING (EFK & PROMETHEUS)

Machine learning models require a blend of traditional infrastructure monitoring and ML-specific observability (data drift, latency profiles) by implementing below.

Metrics & Telemetry: Prometheus and Grafana. Infrastructure Metrics: CPU, GPU utilization, memory footprint, pod restart loops, and queue depths.

Service-Level Objectives (SLOs): P99 inference latency, request error rates (HTTP 5xx) and model-specific prediction errors), and throughput (requests per second).

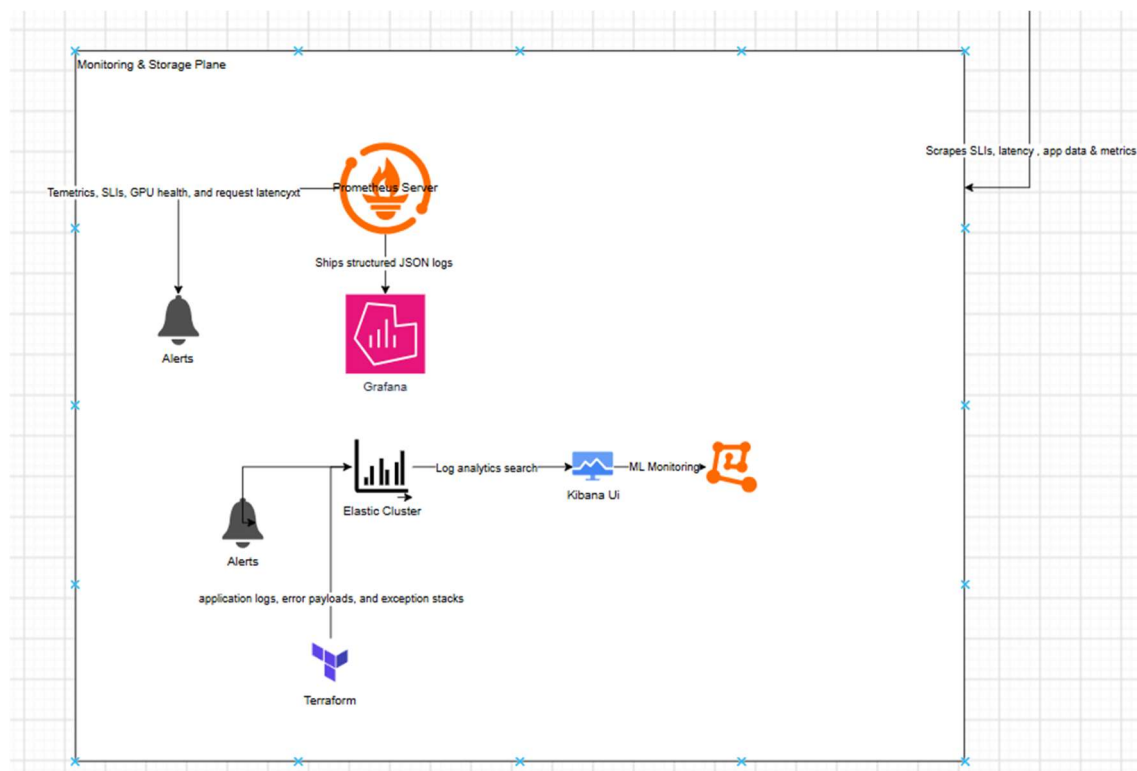
Tracing is critical for multi-model pipelines where a single user request might chain calls across several models.

Logging: Fluentbit or Vector shipping structured JSON container logs to a central backend (such as Grafana Loki or Kquery).

Using Kibana ML model inputs/outputs asynchronously, triggering alerts if feature distributions drift significantly from training baselines. With the ML models in Kibana the monitoring worker calculates drift and operational metrics, expose the results through a Prometheus scrape endpoint. Prometheus can scrape the monitoring worker's /metrics endpoint. Grafana can then visualize the time series and alert on sustained changes. By keeping metric labels bounded. Model name, version and a controlled feature identifier can be suitable, but avoid user IDs, request IDs, raw feature values or unbounded prediction text in labels.

To capture the metrics from the Kubernetes deployments for GPU and for the KubeFlow it can capture exact metrics using the field creation with using painless script.

Implementation of fluentD and the Prometheus exporter with analyzing the cloudwatch and server logs makes much reliable when it comes to the alerts creation in ELK we can implement them using Terraform configurations in order to make them reusable.



9 OPERATIONAL STRATEGIES

Releasing and Updating Model Versions
Model Packaging & Registration: The ML team trains a model, packages it into a Docker image, and pushes it to AWS via automated Jenkins or Github pipelines.

GitOps Manifest Update: A configuration change in Git updates the model's deployment tag (e.g., upgrading from model version v1.1 to v1.2).

Canary Deployments & Traffic Splitting: Using weighted Ingress rules or traffic splits, the platform routes a small percentage of production traffic (e.g., 10%) to the new model version (v1.2) while keeping 90% on v1.1.

Automated Grafana/Prometheus alerts monitor error rates and latency on the new version. **Automated Rollback:** If error rates spike or latency exceeds thresholds on the canary version, traffic is automatically reverted 100% back to the stable version without manual intervention.

Obviously we can configure the default rollback strategy for the Kubernetes in this scenario as well.

As mentioned above with the SLA definitions we can suggest what availability and p99 latency target, and why? For defining the target levels and breaching factors.

Instead of alerting on single error spikes, use Multi-Window Multi-Burn-Rate Alerts to catch rapid error budget depletion without generating false positives. Create Prometheus recording rules to calculate error budget consumption. Configure Grafana Dashboards to visualize live SLI tracking against your SLO targets.